

Running the pricing module on Microsoft Azure

Date: 2026-09-16 · **Status:** a first trial, complete · **Cost so far:** nothing

Who should read what. Sections 1 to 4 are for everyone and assume nothing about cloud computing. **Section 5 is the engineering section** and can be skipped by a finance reader; it is written for the team that will take this to the cloud. **Section 4 contains the only two things we need decided**, and both are yours rather than ours.

This report sits beside the weekly report on the government bonds. They are separate because they have different readers and because the two questions at the end of this one would be buried at the back of the other.

1. What you asked for

"Everyone in our group can run and test these codes."

You asked us to register an Azure account and try it ourselves first. We did. This report says what happened, what it proves, and what it does not.

Short version. The module runs on Azure unchanged. It runs **faster** there than on either machine we normally use. And — the part we thought worth checking carefully — it produces **the same answers**, in a sense we can be precise about rather than assert.

2. What we did, and what it cost

We used **Azure Cloud Shell** — a Linux command line that opens inside the Azure web page, with no software to install on anyone's computer. Two commands: fetch the code, then install and run it.

Automatic checks	495 passed in 38.6 seconds
All four pricing programs	about 40 seconds together; the corporate book takes 7
Cost of the trial	\$0. Cloud Shell is free; it keeps a small amount of storage costing roughly \$0.20 a month
Software we had to install	none beyond the three standard scientific libraries the module already lists

For comparison, **the same 495 checks, from the same version of the code**, take **44.7 seconds** on the development laptop and **228.3 seconds** on the Linux server we deploy to. **Azure is the fastest of the three**, by a clear margin over the server.

⚠ One thing to know about the free account. A new Azure account comes with **\$200 of credit that expires after 30 days**. After that only the permanently-free services remain. Nothing we built will start

charging on its own, but that is worth knowing before anyone builds something larger on this account.

3. Does it give the same answers?

This is the question we spent the most care on, because "the same code" and "the same answers" are not quite the same claim, and the difference matters when numbers go into a report.

3.1 What we checked, and why we checked it in two halves

Every file the module produces has two kinds of column in it, and they deserve different tests:

kind of column	examples	decided by
text	the bond's identifier, which method priced it, its maturity date, any warning attached to it, the reason a bond was set aside	logic
numeric	price, spread, duration, convexity	arithmetic

Text columns must be identical on every machine, because nothing about them involves a calculation. If a bond is routed one way here and another way there, that is a real fault, and it is exactly the kind that hides — it would not show up as an error message, only as a different answer.

Numeric columns are allowed to differ in their last digits, because two different processors do not round identically. That is ordinary and universal; it is not specific to this module or to Azure.

3.2 The result

	outcome
Number of securities in every output file	identical
★ Every text column, in all 13 files	identical, character for character
Numeric columns	agree to better than one part in a million million, except as in 3.3
The spreadsheet interface, checked against 9 worked examples	agrees to within 0.35% of the tolerance we allow

We then repeated the whole thing on a **brand-new machine** a day later and got exactly the same result. A number that reproduces on a fresh machine is a stronger statement than one that reproduced once.

3.3 The one figure that differs, and why it is not an error

Convexity can differ in about the **eighth decimal place** — for example 3.56000000 against 3.56000008.

Here is why, without arithmetic. Convexity describes how a bond's price *accelerates* as interest rates move. To measure it we price the bond three times at slightly different rates and look at how the *change in the change* behaves — a difference of a difference, divided twice by a very small number. That last step multiplies any tiny rounding in the original prices by about **one hundred million**.

So a rounding in the sixteenth decimal place of a price becomes a difference in the eighth decimal place of a convexity. The module is not disagreeing with itself about the bond; it is reporting the last bit of a number through a formula designed to magnify small changes.

Duration, measured the same way but dividing only once, differs around the **twelfth** decimal place.

What this means in practice. If your team compares a file produced on one computer with the same file produced on another, they will find the two are not byte-for-byte identical, and that is expected. **Compare convexity to about seven decimal places and everything else exactly.** Every identifier, route, date and flag should match perfectly — and if one ever does not, that is worth investigating.

3.4 A second finding, which is reassuring in a different way

Azure's copies of the underlying scientific libraries are **substantially newer** than ours — one of them, `pandas`, is a **whole major version ahead**, the kind of upgrade that routinely breaks software.

All 495 checks passed on it anyway, including the ones that reproduce our original bootstrapped curves to nine decimal places and the ones that reconstruct the bond universe to exact counts. The module is not tied to one particular set of library versions, which is one less thing for a cloud migration to worry about.

⚠️ Honest caveat: we have not *pinned* those versions, so that outcome was fortunate rather than guaranteed. Pinning them is a small, sensible piece of housekeeping and is on our list.

4. What "everyone in our group can run" needs from here

The trial answers *can it run there*. It does not by itself answer *can your group use it*. Two things are needed, and both are decisions rather than work.

4.1 Which Azure account should this live in?

Right now it is a **personal trial account** registered on a university email address. It works, but nobody else can be added to it in a way that would make sense for a team.

Whichever account your organisation uses, someone with access to it needs to say so. ⚠️ This is not a question about the data — it is only about where the account lives and who administers it.

(On the data itself: you told us it is not sensitive, and on that basis the full portfolio is what we ran with. Nothing in this report depends on revisiting that.)

4.2 Two different things could be meant by "run and test", and we would rather ask than guess

	what it means	where it stands
A — each person gets their own copy	Anyone on the team opens Azure in a browser, and has a working, private copy of the module they can run, inspect and experiment with	★ Done. This is what the trial built. Set-up is two commands, and after that their environment persists, so returning later is a single command
B — one service everyone calls	The module runs in one place and answers requests over the network, including from Excel — your own "one JSON in, one JSON out" idea	Not built, and a known piece of work. The module was designed with this in mind: the whole interface is already one request in, one answer out, and exactly one file knows about the environment it runs in

Two practical consequences of A, neither a problem but both worth picturing before you choose:

- **Each person needs access to the code repository.** It is private, so whoever you want to give a copy needs to be added to it — a one-off administrative step, but it has to happen before "everyone in our group" is literally true.
- **Each person ends up with their own copy of the holdings file.** That follows from the module carrying its data so it can be run rather than only read. You have said the data is not sensitive; we mention it only so the picture is complete.

These are not alternatives. A is needed either way — the people who would build B need somewhere to build and test it. So the real question is not *which*, but:

Do you want B as well, and on what timeline?

If the answer is yes, the useful next step is small and we can scope it properly once we know which account it lives in.

5. Engineering section

A finance reader can stop here.

5.1 The environment, measured

	development laptop	our Linux server	Azure Cloud Shell
Python	3.13.5	(deployment target)	3.12.14
numpy / pandas / scipy	2.3.4 / 2.3.3 / 1.16.3	—	2.5.3 / 3.0.5 / 1.18.1
495 checks, same commit <code>f2707c3</code>	44.7 s	228.3 s	38.6 s
Text fingerprints vs the release record	(wrote it)	identical	identical, 13 of 13

Fetching the code from GitHub ran at 10–18 MB/s. Worth noting because the same fetch from our Linux server is unreliable — that link is subject to interference, and the Azure link is not. An Azure machine is a *better* GitHub client than our deployment host.

5.2 Verification is a command in the repository, not a procedure

```
PYTHONPATH=src python3 scripts/platform_parity.py --run
```

It runs every pricing program, then compares each output against the published release record on three axes — row count, byte hash, and a **fingerprint over the non-arithmetic columns only** — and finishes by reporting how much of the agreed numerical tolerance the machine consumed.

Two deliberate design choices:

-  **It exits successfully even when files differ.** On a different processor a differing hash is the *expected* reading, and a tool that called that "failure" would train people to ignore the line that actually matters.
- **The fingerprint is computed with the Python standard library only.** Its first version went through `pandas`, and the first machine it met was running `pandas` 3.0 against a record written under 2.3 — every fingerprint differed, and nothing in the output could distinguish *the data changed* from *the serialiser changed*. A check meant to be invariant across machines cannot rest on a library that varies across versions.

The tolerances are **per quantity**: convexity is allowed 1e-6, everything else 1e-10. A single bound looked reasonable and was wrong — sized for convexity's noise, it let a deliberate test perturbation of 1e-4 in a *price* pass unnoticed.

5.3 Four obstacles, none of them in our code

Anyone repeating this will meet all four, in this order:

1. **A new subscription has not enabled the Cloud Shell resource provider.** Register it, then **restart** — storage is attached when a session starts, so registering inside a running session changes nothing for that session.

2. ★ **The storage account must be in the same region as the container.** Cloud Shell places the container near the user; if the storage is elsewhere it silently fails to attach. The fix is to let Azure create the storage account itself.
3. ⚠ **Without attached storage a session is temporary and is erased after about twenty minutes idle.** This is the difference between a demonstration and something a team can use, and it is now configured correctly — home directories and installed packages persist.
4. **The browser terminal corrupts long multi-line pastes.** Put anything substantial in a script in the repository and paste one line. That is why the verification above is a file rather than a command.

5.4 What is already in place for option B

piece	state
A single public entry point, one request in one complete answer out	built
Failures returned as values rather than exceptions	built
The request/response contract, standard library only	built
Exactly one file that knows about its environment	built — a cloud deployment replaces that file and nothing else
A command-line runner, file in / file out	built
An HTTP wrapper, a container definition, and a decision about where the market-data files live	not built — this is the work

6. What we did not do

Said plainly, so that nothing here reads as more than it is:

- We did **not** build a shared environment for your team — only a working one for ourselves, which proves it is possible.
- We did **not** deploy a service. Excel still starts Python on the same computer rather than calling a server.
- We did **not** put anything in an organisation-controlled account.
- We did **not** pin the library versions, so the encouraging result in 3.4 was fortunate as well as real.

Everything measured in this report is measured, not estimated. Every figure comes from a run whose output can be reproduced with the one command in 5.2.